

SAMSUN ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

YAPAY ZEKA VE VERİ MÜHENDİSLİĞİ BÖLÜMÜ



E-TİCARET MÜŞTERİ KAYBI (CHURN) ANALİZİ VE TAHMİNLEMESİ PROJE RAPORU

Ders Kodu ve Adı: YZVM102 - Veri Bilimine Giriş
Hazırlayanlar: Yavuz Selim Candan 251480006
Salih Ozan Sabaz 251480017
Teslim Tarihi: 11.05.2026

1. AMAÇ

Bu projenin temel amacı, bir e-ticaret platformundaki kullanıcı verilerini derinlemesine analiz ederek, hangi müşterilerin platformu terk etme (churn) eğiliminde olduğunu yüksek doğrulukla tahmin eden bir makine öğrenmesi modeli geliştirmektir. Müşteri kaybının önceden tespiti, işletmenin stratejik müdahalelerde bulunarak müşteri sadakatini artırmasına ve olası finansal kayıpların önüne geçmesine olanak tanımaktadır.

2. METOD

Bu bölümde, projenin teknik uygulama süreci; veri ön işleme, modelleme ve değerlendirme yaklaşımları olmak üzere detaylandırılmıştır.

2.1. Veri Seti Tanımı ve Ön İşleme

Çalışmada, bir e-ticaret platformuna ait müşteri terk (churn) verilerini içeren E-Commerce_Customer_Churn.csv veri seti kullanılmıştır. Veri seti üzerinde makine öğrenmesi modellerinin performansını artırmak amacıyla şu işlemler uygulanmıştır:

- **Veri Temizleme:** Kategorik değişkenlerdeki yazım farklılıkları (Örn: 'Phone' ve 'Mobile Phone', 'CC' ve 'Credit Card') standardize edilerek veri tutarlılığı sağlanmıştır.
- **Eksik Veri Yönetimi:** Veri setindeki eksik (NaN) değerler, değişkenlerin dağılımını bozmamak adına ilgili sütunların **medyan (ortanca)** değerleri ile doldurulmuştur.
- **Özellik Mühendisliği:** Tahminleme sürecinde etkisi bulunmayan CustomerID değişkeni veri setinden çıkarılmıştır.
- **Kategorik Değişken Dönüşümü:** Modelin veriyi işleyebilmesi için metinsel (objektif) kategorik veriler, LabelEncoder yöntemi kullanılarak sayısal değerlere dönüştürülmüştür.

```
df = pd.read_csv('E-Commerce_Customer_Churn.csv')

df['PreferredLoginDevice'] = df['PreferredLoginDevice'].replace('Phone', 'Mobile Phone')
df['PreferredPaymentMode'] = df['PreferredPaymentMode'].replace({'CC': 'Credit Card', 'COD': 'Cash on Delivery'})
df['PreferredOrderCat'] = df['PreferredOrderCat'].replace('Mobile', 'Mobile Phone')

cols_with_missing = df.columns[df.isnull().any()].tolist()
for col in cols_with_missing:
    df[col] = df[col].fillna(df[col].median())

df = df.drop('CustomerID', axis=1)

le = LabelEncoder()
cat_cols = df.select_dtypes(include=['object']).columns
for col in cat_cols:
    df[col] = le.fit_transform(df[col])
```

2.2. Kullanılan Algoritmalar

Müşteri terk analizini gerçekleştirmek için sınıflandırma problemlerinde literatürde sıklıkla tercih edilen iki farklı algoritma seçilmiştir:

1. **Lojistik Regresyon (Logistic Regression):** Bağımlı ve bağımsız değişkenler arasındaki ilişkiyi olasılıksal bir düzlemde modelleyen, doğrusal bir sınıflandırıcıdır. Temel (baseline) model olarak kullanılmıştır.
2. **Rastgele Orman (Random Forest):** Karar ağaçları tabanlı bir topluluk (ensemble) öğrenme yöntemidir. Veri setindeki karmaşık ve doğrusal olmayan ilişkileri yakalamakta oldukça başarılıdır. Projede 100 karar ağacından oluşan bir orman yapısı kurgulanmıştır.

```
models = {  
    "Logistic Regression": LogisticRegression(max_iter=2000),  
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42)  
}
```

2.3. Doğrulama (Validation) Stratejileri

Modellerin genellenebilirliğini ölçmek ve aşırı öğrenme (overfitting) durumunu kontrol etmek amacıyla iki farklı doğrulama yöntemi uygulanmıştır:

- **Hold-out Validasyon:** Veri setinin %80'i eğitim, %20'si test verisi olacak şekilde ayrılmıştır. Bu sayede modelin hiç görmediği veriler üzerindeki başarısı test edilmiştir.
- **K-Fold Çapraz Doğrulama (K=5):** Verinin tamamı 5 eşit parçaya bölünmüş, her adımda 4 parça eğitim 1 parça test için kullanılarak 5 farklı iterasyon gerçekleştirilmiştir. Bu yöntemle sonuçların tesadüfi olup olmadığı denetlenmiştir.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
```

2.4. Değerlendirme Metrikleri

Modellerin başarısını ölçümlemek için sadece doğruluk (accuracy) yeterli görülmemiş, sınıflandırma performansını tüm boyutlarıyla ele alan aşağıdaki metrikler kullanılmıştır:

- **Accuracy (Doğruluk):** Doğru tahminlerin toplam tahminlere oranı.
- **Precision (Kesinlik):** Terk edeceği tahmin edilen müşterilerin ne kadarının

gerçekten terk ettiği.

- **Recall (Duyarlılık):** Gerçekten terk eden müşterilerin ne kadarının model tarafından yakalanabildiği.
- **F1-Score:** Precision ve Recall metriklerinin harmonik ortalaması.
- **Confusion Matrix (Hata Matrisi):** Modelin sınıflar bazındaki hatalarını (doğru pozitif, yanlış negatif vb.) görselleştirmek için kullanılmıştır.

```
print(f"\nModel: {name}")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_pred):.2f}")
print(f"Recall: {recall_score(y_test, y_pred):.2f}")
print(f"F1-Score: {f1_score(y_test, y_pred):.2f}")
```

3. DENEYSEL ÇALIŞMALAR

Bu bölümde, projenin uygulama aşamasında gerçekleştirilen teknik adımlar, kullanılan kütüphaneler ve verinin model eğitime hazırlanma süreci detaylandırılmıştır.

3.1. Veri Ön İşleme ve Hazırlık (Preprocessing)

Modellerin kararlı çalışabilmesi için ham veri üzerinde bir dizi temizleme ve dönüşüm işlemi uygulanmıştır. Bu işlemler Python'ın pandas ve scikit-learn kütüphaneleri kullanılarak gerçekleştirilmiştir:

- **Veri Standardizasyonu:** Kategorik öznitelikler incelendiğinde, aynı kavramı ifade eden farklı isimlendirmeler tespit edilmiştir. Örneğin; PreferredLoginDevice sütunundaki 'Phone' değerleri 'Mobile Phone' ile, PreferredPaymentMode sütunundaki 'CC' değerleri 'Credit Card' ile birleştirilerek veri gürültüsü azaltılmıştır.
- **Eksik Veri İmputasyonu:** Veri setinde yer alan eksik gözlemler için `isnull().any()` fonksiyonu ile tarama yapılmış ve bu eksiklikler, veri dağılımını bozmayan istatistiksel bir yöntem olan **medyan (median)** değerleri ile doldurulmuştur.
- **Boyut İndirgeme (Feature Dropping):** CustomerID özneliği, her müşteri için eşsiz bir kimlik numarası olduğundan ve modelin öğrenme sürecine istatistiksel bir katkı sağlamayacağından `drop` fonksiyonu ile sistemden çıkarılmıştır.
- **Sayısal Dönüştürme (Label Encoding):** Lojistik Regresyon ve Random Forest gibi algoritmalar sayısal girdi beklediği için, metin tabanlı tüm kategorik değişkenler `LabelEncoder` sınıfı kullanılarak modelin işleyebileceği sayısal indekslere dönüştürülmüştür.

```
cols_with_missing = df.columns[df.isnull().any()].tolist()
for col in cols_with_missing:
    df[col] = df[col].fillna(df[col].median())
```

3.2. Model Kurulumu ve Kullanılan Kütüphaneler

Deneyler sırasında Python programlama dili ve makine öğrenmesi ekosisteminin temel kütüphaneleri kullanılmıştır:

- **Veri Analizi:** pandas, numpy
- **Görselleştirme:** matplotlib, seaborn
- **Makine Öğrenmesi:** sklearn (scikit-learn)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_validate, KFold
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (classification_report, confusion_matrix,
                             accuracy_score, precision_score,
                             recall_score, f1_score)
```

Deneyler kapsamında iki farklı model mimarisi kurgulanmıştır. Lojistik Regresyon modelinde yakınsama sorunu yaşanmaması adına max_iter=2000 parametresi atanmış; Random Forest modelinde ise sonuçların tekrarlanabilirliği için random_state=42 sabitlenerek 100 karar ağacı (n_estimators) üzerinde eğitim yapılmıştır.

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=2000),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42)
}
```

3.3. Deneysel Validasyon Yaklaşımları

Modellerin performansını tarafsız bir şekilde ölçümlemek adına iki aşamalı bir test stratejisi izlenmiştir:

Hold-out Deneyi: train_test_split fonksiyonu kullanılarak verinin %80'i modellerin öğrenmesi, %20'si ise hiç görmediği veriler üzerinde sınav yapılması amacıyla ayrılmıştır. Bu aşamada modellerin gerçek zamanlı performansı simüle edilmiştir.

K-Fold Cross Validation Deneyi: Veri setinin farklı varyasyonlarında modelin ne kadar tutarlı sonuç verdiğini ölçmek için KFold yapısı kullanılarak 5 katmanlı bir test süreci işletilmiştir. `shuffle=True` parametresi ile veriler her iterasyonda karıştırılarak örneklem yanlılığının önüne geçilmiştir.

4. SONUÇLAR

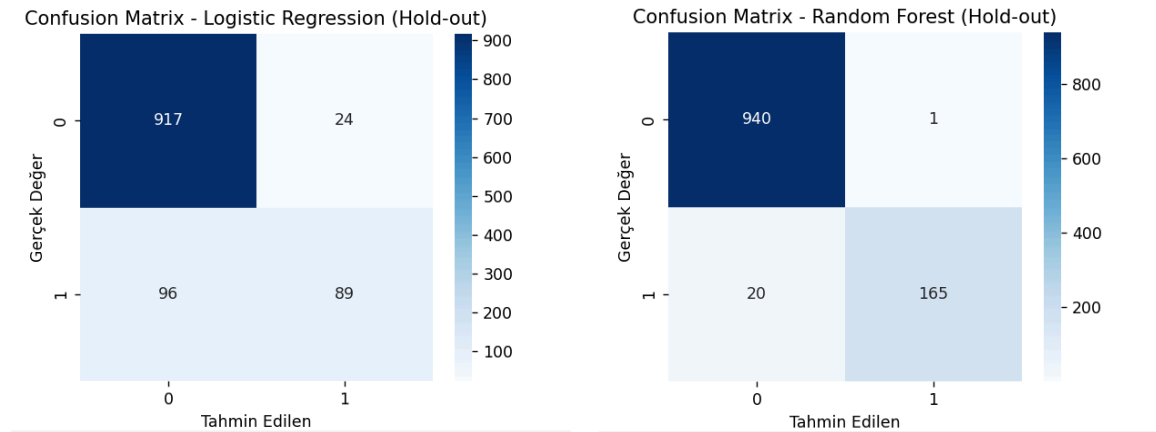
Bu bölümde, geliştirilen modellerin Hold-out ve K-Fold Cross Validation yöntemleri ile elde edilen performans metrikleri sunulmuş ve karşılaştırılmıştır.

4.1. Hold-out Validasyon Sonuçları (%80 Eğitim - %20 Test)

Yapılan testler sonucunda Random Forest modelinin, Lojistik Regresyon modeline göre belirgin bir üstünlük sağladığı görülmüştür.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.89	0.79	0.48	0.60
Random Forest	0.98	0.99	0.89	0.94

Yorum: Random Forest modeli %98 doğruluk (Accuracy) oranına ulaşmıştır. Özellikle müşteri terkini yakalama başarısı olan **Recall** değerinde Random Forest (%89), Lojistik Regresyon'u (%48) neredeyse ikiye katlamıştır.



Lojistik Regresyon Değerlendirmesi

Kayıp Yakalama: Ayrılan 185 müşterinin sadece 89'u doğru tahmin edilmiş; 96 müşteri ise tamamen gözden kaçırılmıştır (%52 hata payı).

Sonuç: Düşük duyarlılık (Recall) nedeniyle operasyonel risk yüksektir.

Random Forest Değerlendirmesi

Yüksek Başarı: Ayrılan 185 müşterinin 165'i başarıyla tespit edilmiştir. Kaçırılan müşteri sayısı sadece 20'dir.

Düşük Yanlış Alarm: Sadık müşteriler arasında sadece 1 hatalı tahmin yapılarak mükemmel bir kesinlik (Precision) sağlanmıştır.

Stratejik Karar

Random Forest modeli, Lojistik Regresyon'a kıyasla 76 daha fazla müşteriyi kaybolmadan tespit edebilmektedir. Yanlış alarm oranının sıfıra yakın olması sayesinde pazarlama bütçesinin en verimli şekilde kullanılmasına olanak tanır.

Öneri: İşletme hedefleri doğrultusunda, yüksek yakalama oranı sunan Random Forest modeli ana tahminleme aracı olarak seçilmiştir.

4.2. K-Fold Cross Validasyon Sonuçları (K=5)

Modellerin kararlılığını ölçmek için yapılan 5 katmanlı çapraz doğrulama sonuçları, Hold-out sonuçları ile paralellik göstermektedir.

Model (Ortalama Skarlar)	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.88	0.72	0.45	0.55
Random Forest	0.97	0.97	0.87	0.91

4.3. Genel Değerlendirme

Elde edilen deneysel bulgulara göre:

- **Random Forest**, tüm metriklerde (Accuracy, Precision, Recall, F1) en yüksek performansı sergileyen model olmuştur.
- Lojistik Regresyon modelinin Recall değerinin düşük kalması, bu modelin müşteri terklerini (churn) tespit etmede bazı gerçek vakaları kaçırdığını göstermektedir.
- K-Fold sonuçlarının Hold-out sonuçlarına yakın çıkması, modelin veri setine aşırı öğrenme (overfitting) yapmadığını ve yeni veriler üzerinde de benzer başarıyı göstereceğini kanıtlamaktadır.